PocketFin

Jellyfin on your Analogue Pocket



Every page of the documentation, printed from the repository's Markdown.
For the interactive version, download the repository and double-click Open-Documentation.html.

Created by ChaoticSPARTAN1

Contents

START

Start here ..... README.md
Wiki home ..... docs/Home.md
What you need ..... docs/What-You-Need.md
Install ..... docs/Install.md
First run ..... docs/First-Run.md

GUIDE

Using PocketFin ..... docs/Using-PocketFin.md
Jellyfin wirelessly ..... docs/Wireless-Jellyfin.md
Offline library ..... docs/Offline-Library.md

HELP

Troubleshooting and recovery ..... docs/Troubleshooting.md
FAQ ..... docs/FAQ.md

MORE

Building from source ..... docs/Building-from-Source.md
AI prompt examples ..... docs/AI-Prompt-Examples.md
Credits and licences ..... docs/Credits-and-Licences.md
Contributing ..... CONTRIBUTING.md
Security ..... SECURITY.md
Third-party notices ..... THIRD-PARTY-NOTICES.md
Additional terms ..... ADDITIONAL-TERMS.md

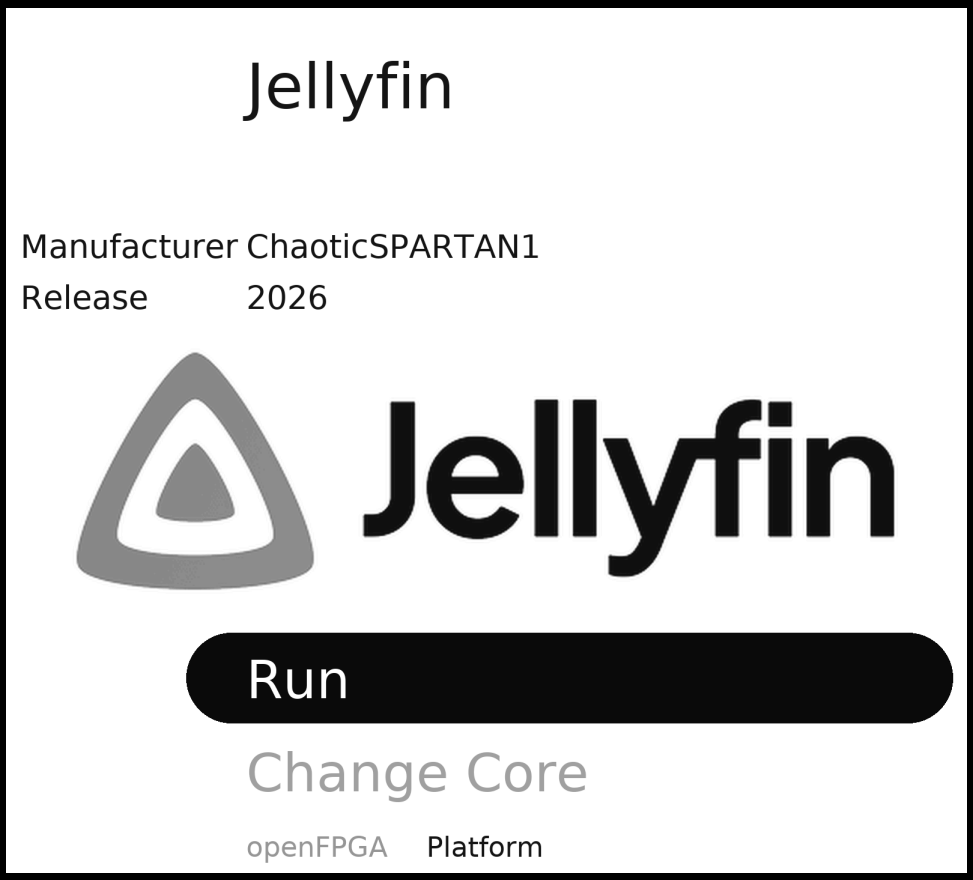
PocketFin / README.md

PocketFin

Jellyfin on your Analogue Pocket

PocketFin: Jellyfin on your Analogue Pocket

Read the guide: docs/PocketFin-Guide.pdf (opens right on GitHub), or download and double-click `Open-Documentation.html` for the interactive version.



The core's launch screen, redrawn from the core's own files. Jellyfin logo (c) the Jellyfin contributors, CC BY-SA 4.0.

PocketFin is an openFPGA core and app that brings a Jellyfin-style media library to the Analogue Pocket. It plays video with sound using a JPEG decoder built into the FPGA. On the Pocket it appears as the core **Jellyfin** (made by ChaoticSPARTAN1).

Watch videos straight from the SD card, no network or extra hardware needed. Put your own videos on the card and they show up in the **On this Pocket** tile on the Home screen.

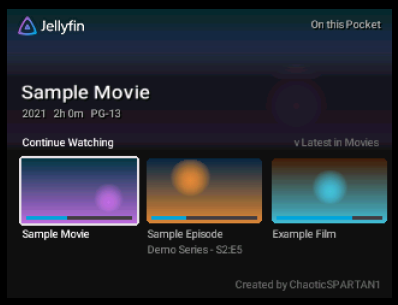
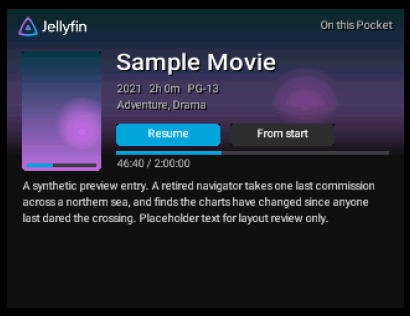
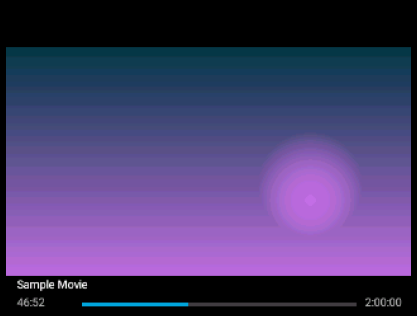
Add your own videos to the card (short version)

1. Install PocketFin on the card first (Install).
2. On a Windows PC, install Python 3.12+, FFmpeg (for example `winget install Gyan.FFmpeg`) and PyAV: `py -3 -m pip install av`. FFmpeg and PyAV are separate free software under their own licences (FFmpeg: LGPL or GPL depending on the build; PyAV: BSD) and are not included in PocketFin. See What you need.
3. Download this repository (green **Code** button, **Download ZIP**) and unzip it.
4. In that folder, run: `py -3 tools\make_offline_library.py --video "C:\Videos\My Video.mp4"` (add more `--video "..."` for more files).
5. It converts each video to the Pocket's format (256x144 pictures at the video's own frame rate, about 0.9 GB per hour) and writes the result to `build\offline-library\`.
6. Copy everything inside `build\offline-library\` onto the root of the Pocket's SD card and allow it to replace files. Its files go to `Assets\pocketlive\common\` and `Assets\pocketlive\ChaoticSPARTAN1.Jellyfin\`.
7. Put the card back, open the Jellyfin core and press **A**: your videos are in **On this Pocket**.

Limits: up to 256 videos, 12 pack files of 2 GiB each (about 24 hours in total). Only add videos you own or have the right to copy. Details and troubleshooting: Offline library.

What it does

- **Jellyfin-style screens**: Home rows (Continue Watching, Next Up, Latest), My Media, details pages, posters and a resume bar, in the style of the official Jellyfin apps.
- **Offline library on the SD card**: your own videos, or titles copied from your Jellyfin server.
- **Live Jellyfin over the link cable**: the Pocket has no Wi-Fi, but with a ModRetro Chromatic running Jellymatic and a link cable, the Pocket browses your server and streams from it (how).
- **Each title at its own frame rate** (24, 25 or 30 fps), resume where you stopped, watched marks.

Home	Details	Playing
		

Screenshots come from the project's own screen renderer with made-up sample titles.

Quick start

1. Download this repository and unzip it.
2. Copy the contents of `release\SD\` onto the root of your Pocket's SD card. Then, **first install only (skip if the card already has them)**, also copy `release\SD-sample-library\` and `release\SD-first-install-saves\`. Or, on Windows, run `.\Install-PocketFin.ps1 -Card E:\` (use your card's drive letter). Full steps.
3. On the Pocket: **Openfpga** (or the core list) > **Jellyfin** > **Run**. Press **A** and play the built-in **Test Pattern** to check picture and sound.

Documentation

The docs cover everything in plain steps: What you need, Install, First run, Using PocketFin, Connecting to Jellyfin wirelessly, Offline library, Troubleshooting and recovery, FAQ, Building from source, AI prompt examples, Credits and licences.

As one PDF: docs/PocketFin-Guide.pdf holds every page, with a clickable table of contents; GitHub shows it right in the browser.

Offline: download or clone this repository, then double-click `Open-Documentation.html` in its main folder. It opens all of these pages in your browser, with pictures, without a server or an internet connection (keep it in the repository folder: it loads the pictures from `docs/`). It is built from the Markdown pages by `tools/build_docs_html.py` (the PDF by `tools/build_docs_pdf.py`).

Optional: Remote Pilot for the Chromatic side

Remote Pilot is a free PC companion for the Chromatic (with Jellymatic): it shows the Chromatic's screen in your web browser and lets you press its buttons and read its logs from the PC, over USB or Wi-Fi. For PocketFin users it makes the Chromatic side easier: set up its Wi-Fi, Jellyfin sign-in and Pocket link from your desk, and troubleshoot the link without juggling two handhelds. Its privacy eye (on by default) hides addresses and network names on screen, which helps when you make a bug report, and an AI assistant such as Claude or Codex can use it to help you test. It is optional: you do not need it to play videos from the SD card. More: Connecting to Jellyfin wirelessly.

Related projects

- Jellymatic: Jellyfin on the ModRetro Chromatic, and the Chromatic side of the link-cable connection.
- RemotePilot: see and control a Chromatic from your PC (screen, buttons, logs) over USB or Wi-Fi.

Status

Version 1.0.0. The app in `release/` (`pjlive.elf`) was built from this source. The bitstream and OS kernel are the files that have run on an Analogue Pocket (only `core.json` gained the version number and project link). The previous app build (SHA-256 `a19aa677...`) was tested on a Pocket; this build changes only the About screen's text and the source of the font atlas, which is now generated directly from Google's Roboto font (same glyph sizes and spacing; only tiny anti-aliasing differences). Live streaming through a Chromatic works but is limited by the link cable: pictures are small and soft, and playback may pause briefly to refill. See the FAQ.

Disclaimer

PocketFin is an independent project, not affiliated with, endorsed by or supported by Analogue or the Jellyfin project. Analogue and Analogue Pocket are trademarks of Analogue Enterprises Limited. Jellyfin is a trademark of its owners; the Jellyfin logo is used under CC BY-SA 4.0. ModRetro and Chromatic are trademarks of ModRetro, Inc.; PocketFin is not affiliated with ModRetro either. Use it at your own risk. It is provided as is, without warranty, and support is not guaranteed.

Contributing

Bug reports, fixes and pull requests are welcome: see CONTRIBUTING.md. Report security problems privately: see SECURITY.md.

Licence

PocketFin is free software: GPL-3.0-or-later (LICENSE), with additional terms and one additional permission under GPL section 7 (ADDITIONAL-TERMS.md):

- keep the author attribution "ChaoticSPARTAN1" in the credits and in any About or credits screen (section 7(b)), and do not misrepresent the origin of the work (section 7(c));
- you may convey the FPGA bitstream, which combines PocketFin with the Analogue Pocket Framework and Intel/Altera FPGA IP; its Corresponding Source need not include the source of those parts.

Third-party parts keep their own licences (THIRD-PARTY-NOTICES.md).

Credits

- **Created by ChaoticSPARTAN1.**
- openfpgaOS runtime and SDK by ThinkElastic (Apache-2.0).
- core_jpeg hardware JPEG decoder by Ultra-Embedded.com (Apache-2.0).
- VexiiRiscv CPU by SpinalHDL (MIT).
- Roboto font by Google (Apache-2.0); the app's glyph atlas is generated from it by `tools/build_roboto_atlas.py`.
- musl libc, dlmalloc, lib9660 and cram0_phy by their authors (MIT, MIT-0, ISC, MIT).
- Jellyfin logo by the Jellyfin contributors (CC BY-SA 4.0).
- Analogue Pocket Framework by Analogue.

Full list: Credits and licences.

PocketFin / docs/Home.md

PocketFin wiki



PocketFin

PocketFin puts a Jellyfin-style media library on the Analogue Pocket. It runs as an openFPGA core called **Jellyfin**. You can watch videos stored on the SD card with nothing else, or browse and stream your Jellyfin server through a ModRetro Chromatic and a link cable.

Read it as one PDF: PocketFin-Guide.pdf (opens right on GitHub). **Or offline:** download or clone the repository, then double-click `Open-Documentation.html` in its main folder. Every page here opens in your browser, with pictures, with no internet connection needed.



The core's launch screen. Jellyfin logo (c) the Jellyfin contributors, CC BY-SA 4.0.

Pages

Page	What it covers
What you need	The Pocket, an SD card, and what is optional
Install	Copying PocketFin onto the SD card, updating, removing
First run	Starting the core and playing the test pattern
Using PocketFin	Buttons, Home rows, On this Pocket, playback, quality
Connecting to Jellyfin wirelessly	The Chromatic link and setting up a Jellyfin server at home
Offline library	Putting your own videos (or server titles) on the card
Troubleshooting and recovery	Black screen, flashing backlight, going back to the previous files
FAQ	Short answers
Building from source	Rebuilding the app, the core and the release
AI prompt examples	Ready-made prompts for Claude, Codex and other assistants
Credits and licences	Who made what, and under which licence

Related projects

- Jellymatic: Jellyfin on the ModRetro Chromatic and the Chromatic side of the link cable.
- RemotePilot: an optional PC companion that shows and controls the Chromatic from your PC (why it helps).

Disclaimer

PocketFin is an independent project, not affiliated with, endorsed by or supported by Analogue, ModRetro or the Jellyfin project. Jellyfin is a trademark of its owners. Use it at your own risk; support is not guaranteed.

What you need

- To watch videos from the card
- To add your own videos
- To use your Jellyfin server live

To watch videos from the card

- An **Analogue Pocket** with current firmware (openFPGA support; firmware 2.2 or newer).
- Its **SD card**, and a way to copy files to it: a card reader on your computer, or the Pocket's own USB card access if your firmware offers it.
- About **5 MB** free for the core and demo, plus room for your videos (about 0.9 GB per hour).

That is all. No network, no Chromatic, no server.

To add your own videos

A Windows PC (other systems work for the tools, but the steps here are written for Windows) with:

- **Python 3.12 or newer**: https://www.python.org/downloads/ (tick "Add python.exe to PATH" when installing). The commands below use `py -3`.
- **FFmpeg** (version 7 or newer) with `ffmpeg.exe` and `ffprobe.exe` on your PATH. One easy way: `winget install Gyan.FFmpeg`, then open a new terminal. FFmpeg is separate software under its own licence (LGPL or GPL depending on the build); PocketFin does not include it.
- **PyAV**: `py -3 -m pip install av`.

See Offline library.

To use your Jellyfin server live

- A **Jellyfin server** at home (free; see Connecting to Jellyfin wirelessly).
- A **ModRetro Chromatic** with **Jellymatic** installed: https://github.com/ChaoticSPARTAN1/Jellymatic
- A **link cable** that fits the link ports of both consoles.
- A 2.4 GHz Wi-Fi network for the Chromatic.

Next: Install

PocketFin / docs/Install.md

Install on the Pocket

Home > Install

- What is in release/
- Install by copying files
- Install with the Windows script
- Update to a new version
- Remove PocketFin

PocketFin adds its own folders to the card. It does not change other cores, games, saves or settings.

What is in release/

Folder	What it holds	When to copy it
`release/SD/`	The core (`Cores/ChaoticSPARTAN1.Jellyfin/`), the app and OS (`Assets/pocketlive/common/`), the platform entry and the licence notices	First install and every update
`release/SD-sample-library/`	The demo library (a 20-second test pattern) and the file that tells the Pocket which library files to load	First install only, or to go back to the demo
`release/SD-first-install-saves/`	Empty save files for the app's log and offline cache (`Saves/pocketlive/common/`)	Only if your card does not have them yet

`release/SHA256SUMS.txt` lists a checksum for every file.

Install by copying files

1. Turn the Pocket off and take out the SD card (or use the Pocket's USB card access). Put the card in your computer.

2. **Back up the card** (copy everything on it to a folder on your computer). This is your way back if anything goes wrong.
3. Open `release/SD/` and copy everything inside it (the folders `Assets`, `Cores`, `Notices`, `Platforms`) onto the root of the card. Say yes if asked to merge folders.
4. **First install only (skip if the card already has them):** do the same with everything inside `release/SD-sample-library/`.
5. **First install only (skip if the card already has them):** do the same with everything inside `release/SD-first-install-saves/`, but **skip any file that already exists** (choose "Skip these files"). Never replace existing saves.
6. Eject the card safely, put it back in the Pocket and see First run.

Do not change any file in `Cores/ChaoticSPARTAN1.Jellyfin/`. In particular, never edit `interact.json`: other versions of that file made the Pocket hang at a black screen.

Install with the Windows script

`Install-PocketFin.ps1` does the same steps and checks every file:

```
.\Install-PocketFin.ps1 -Card E:\            # use your card's drive letter
.\Install-PocketFin.ps1 -Card E:\ -WhatIf    # only show what it would do
```

It checks the release against `SHA256SUMS.txt`, copies `release/SD/`, adds the demo library only if the card has no PocketFin library, creates save files only where none exist, and reads every copied file back to check it. Any card file it replaces is first copied to `backups\<date-time>\` next to the script. It never needs a password or address and sends nothing anywhere.

If Windows refuses to run the script, run this once in the same window first: `Set-ExecutionPolicy -Scope Process Bypass`.

Update to a new version

Copy only `release/SD/` (or run the script again). Your library, your saves and the list of library files stay as they are. Do not copy `SD-sample-library` over your own library unless you want the demo back.

Remove PocketFin

By hand, delete these from the card:

- `Cores/ChaoticSPARTAN1.Jellyfin/`
- `Assets/pocketlive/ChaoticSPARTAN1.Jellyfin/`
- `os.bin`, `pjlive.elf` and `pjlive.ini` in `Assets/pocketlive/common/`
- `Platforms/pocketlive.json` and `Platforms/_images/pocketlive.bin`
- `Notices/PocketLive/`
- Optional, your library and saves: the rest of `Assets/pocketlive/common/` and `Saves/pocketlive/`

Or run `.\Install-PocketFin.ps1 -Card E:\ -Remove` (add `-RemoveLibraryAndSaves` to remove those too). The script moves everything to `backups\` on your PC instead of deleting it.

Next: First run

First run

- Start the core
- The start screen
- Play the test pattern
- Quitting

Start the core

1. Turn on the Pocket.
2. Open **openFPGA** in the main menu (on some firmware it is the core list), choose **Jellyfin**, then **Run**.



It takes a few seconds to load. The app tests its own JPEG decoder at every start.

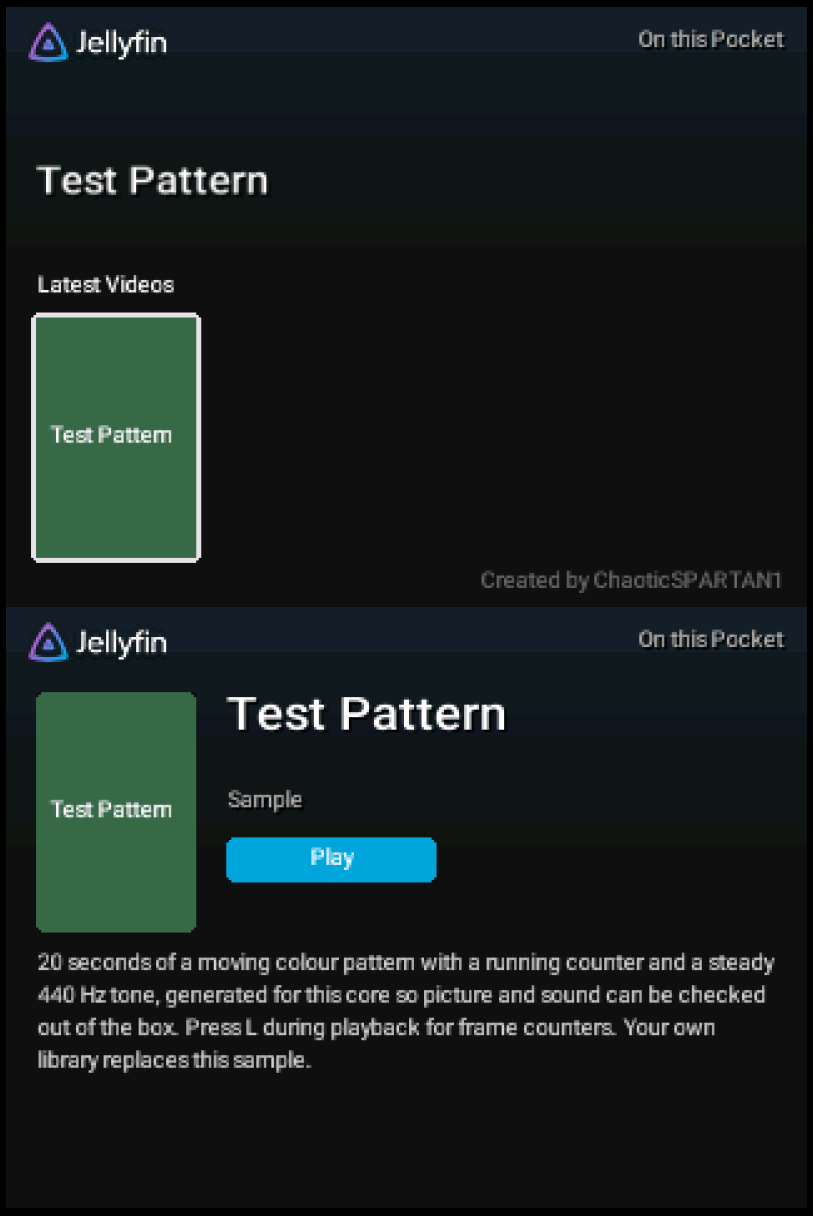
The start screen



- The top lines show the self-test: **Runtime: ok** and **FPGA JPEG decoder: ok** mean the core works. **Library on card** shows how many titles are on the card.
- **A** opens a library: the one on this Pocket, or the one saved from your Chromatic if you have used one before (then **Select** opens the card's library instead).
- **X** looks for a Chromatic on the link cable now. It also looks by itself every few seconds.
- **Y** shows **About and credits**.

Play the test pattern

1. Press **A**. Home opens with the built-in **Test Pattern**.
2. Press **A** on it to open its details, then **A** again to play.
3. You should see moving colour bars with a running counter and hear a steady tone. Press **L** to show frame counters, **B** to stop.



If picture and sound work, the core is installed correctly. Next, add your own videos (Offline library) or connect to your server (Wireless).

Quitting

Use the Pocket's own menu (the Analogue button) and quit the core. **B** on the start screen does not quit; it shows that hint instead. Quitting properly lets the Pocket save the app's log, where you stopped each title and the saved Chromatic library to the card.

Next: Using PocketFin

Using PocketFin

- Buttons
- Home
- On this Pocket
- Details and playback
- Frame rate and picture quality
- Settings in the Pocket's menu

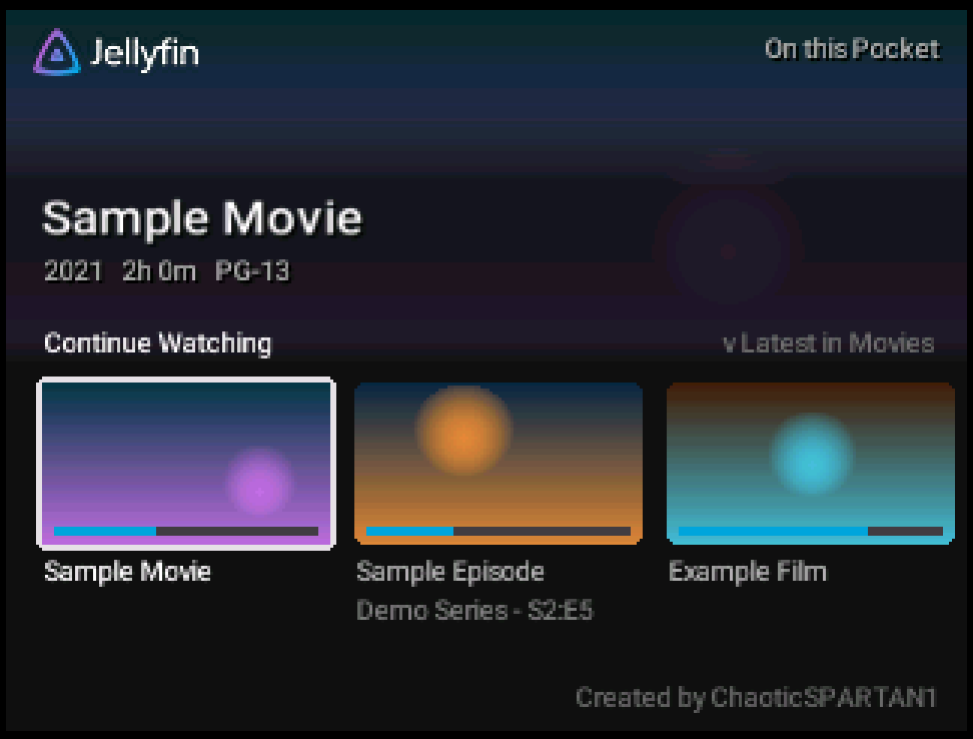
Buttons

Where	Button	What it does
Start screen	A	Open a library (this Pocket's, or the one saved from your Chromatic)
	Select	Open this Pocket's library when a saved Chromatic library also exists
	X	Look for a Chromatic on the link cable now
	Y	About and credits (B or Y returns)
Home and folders	D-pad	Move between rows (up/down) and titles (left/right)
	A	Open the title or folder
	X	Play the selected title from the start
	B	Back
	Y	Go to Home
	L / R	Previous / next page in a large folder

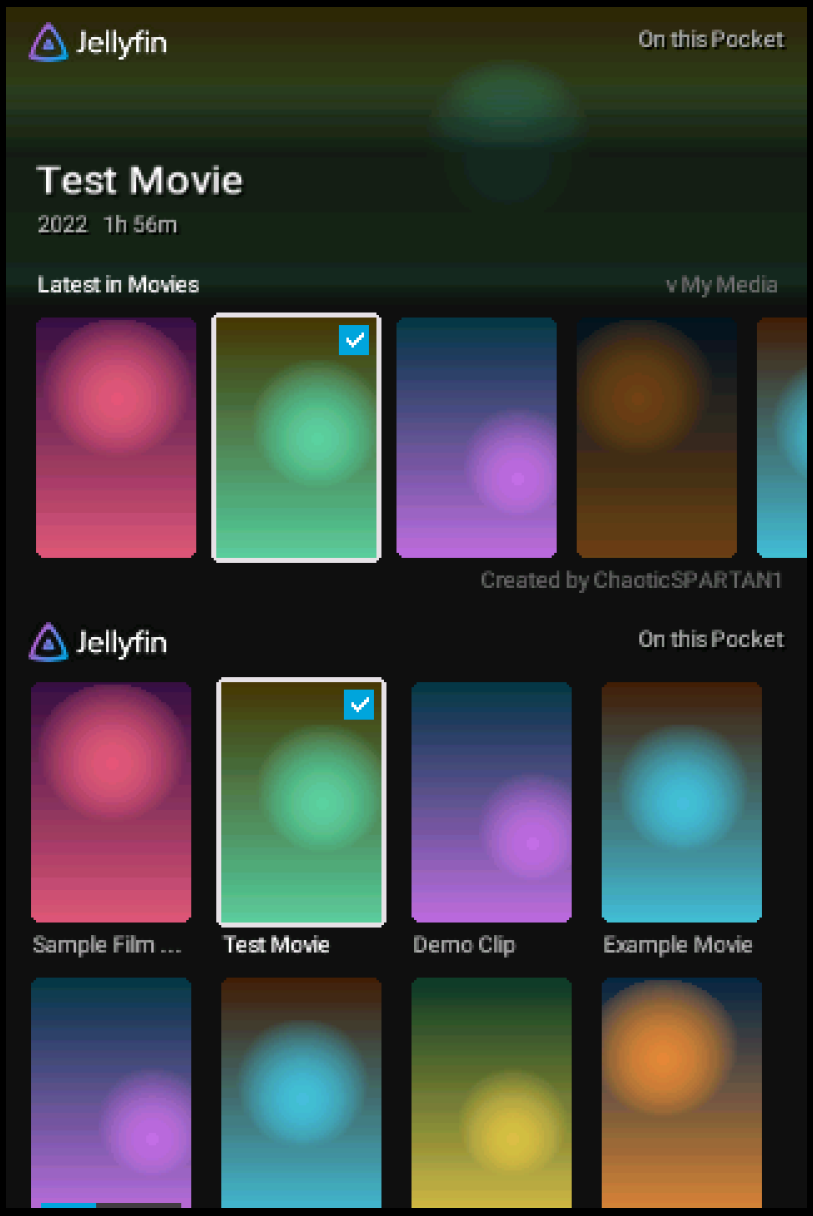
Where	Button	What it does
	Select	With this Pocket's library: back to the start screen. With a Chromatic: change the frame-rate limit
Details	A	Play, or Resume / From start (choose with left/right)
	X	Play from the start
	B	Back
While playing	Start	Pause; Start or A resumes
	Left / Right	Jump back / forward 10 seconds
	B	Stop (the position is kept for Resume)
	L	Show playback numbers, then (with a Chromatic) link numbers, then hide
	R	Film mode on/off: the screen refresh follows the video's frame rate

To quit the core, use the Pocket's own menu (Analogue button).

Home



Home works like the Jellyfin apps: a large picture of the selected title at the top, then rows such as **Continue Watching**, **Next Up**, **Latest in Movies** and **My Media** (your libraries as tiles). Titles you started have a progress bar; watched titles have a tick. The top right says where the library comes from: **On this Pocket**, or **Chromatic connected**.

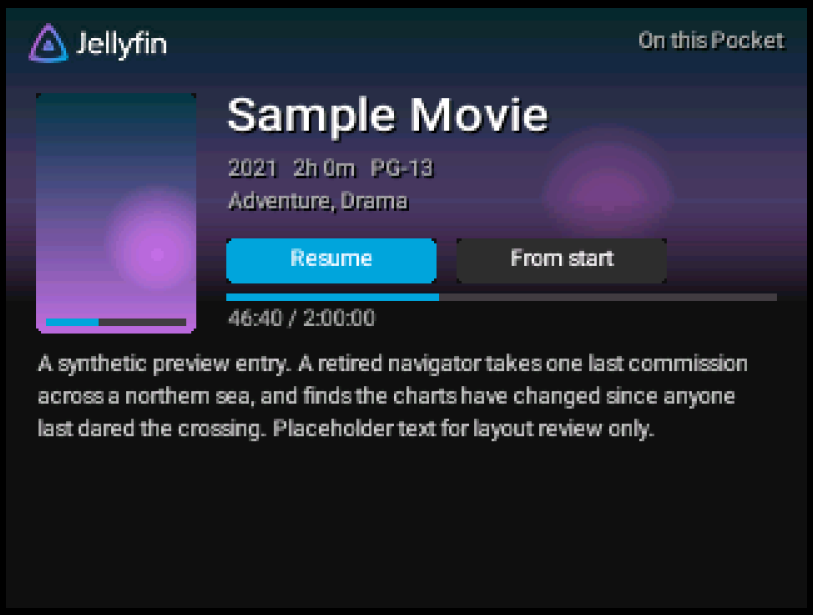


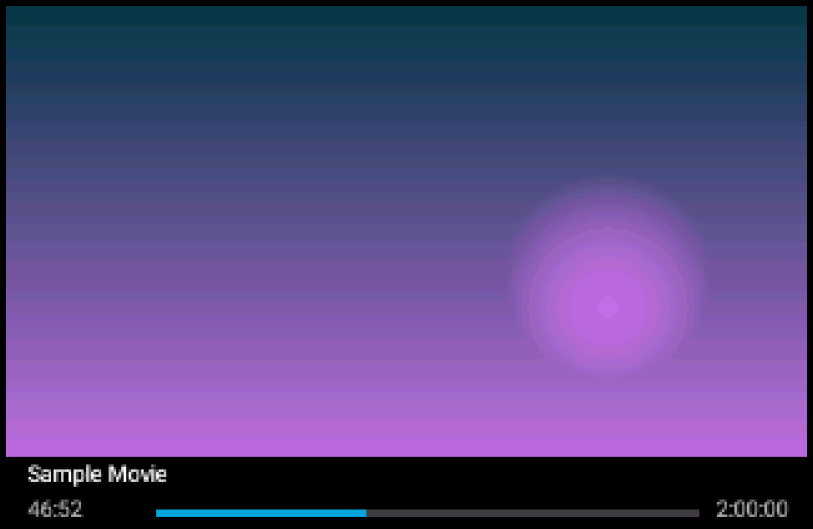
On this Pocket

Everything stored on the SD card opens from the **On this Pocket** tile, at the end of the **My Media** row. It is there both when you use the card on its own and when a Chromatic is connected, so your own videos are always one tile away. Titles that are listed but not stored on the card say **Not on this Pocket**.



Details and playback





The details page shows the year, length, rating, genres and description, with **Resume** and **From start** when you stopped part way. During playback a strip shows the title and progress for a few seconds after each button press. Where you stopped and what you watched are saved on the Pocket and written to the card when you quit the core.

Frame rate and picture quality

- **From the card**: each title plays at its own frame rate (23.976, 24, 25, 29.97 or 30 fps) at 256x144. Faster videos were halved when they were converted.
- **From a Chromatic**: the picture comes from your server through the link cable, so it is smaller (about 128x72) and softer. **Select** cycles the frame-rate limit between 24, 30 and 60 fps; a 24 fps film still plays at 24. The link may not carry the higher rates; the app then says what rate it plays at. See Connecting to Jellyfin wirelessly.

Settings in the Pocket's menu

The Pocket's **Core Settings** and **Controls** screens for this core still show labels left over from the openFPGA template it is built on (for example Analogizer options and game-style button names). PocketFin ignores them; leave them as they are. The app uses its own defaults: link speed Auto (384 kHz, falling back to 256 kHz), a 2-second buffer, normal volume.

Next: Connecting to Jellyfin wirelessly

Connecting to Jellyfin wirelessly

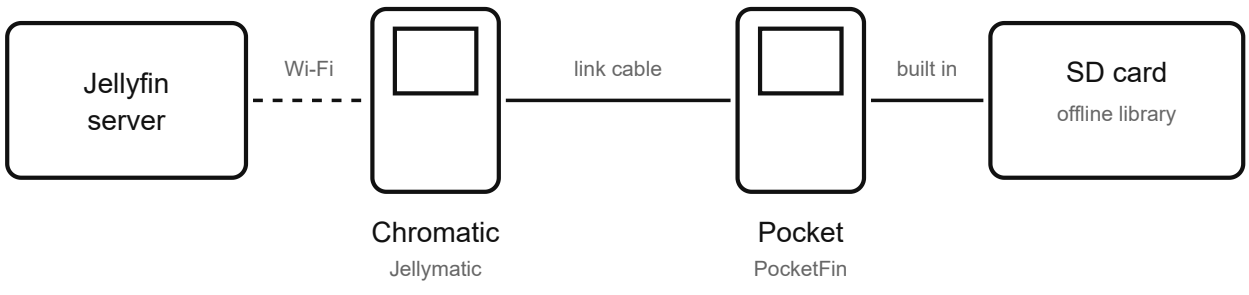
- The two ways to get your library
- Set up a Jellyfin server at home
- Connect through a Chromatic
- What to expect
- Optional: Remote Pilot
- Privacy

The two ways to get your library

The Analogue Pocket has **no Wi-Fi**. PocketFin supports two ways to use your Jellyfin server:

1. **Live, through a ModRetro Chromatic.** The Chromatic joins your Wi-Fi and talks to your server; the Pocket talks to the Chromatic through a link cable. You browse your server's Home, folders and posters on the Pocket and stream titles. The Chromatic needs Jellymatic.
2. **Copied to the card from a PC.** `tools/make_offline_library.py --jellyfin` copies titles from your server to the SD card, with their posters and descriptions. No wireless at all; see Offline library.

There is no direct PC-to-Pocket wireless connection.



1 Live: server → Wi-Fi → Chromatic → link cable → Pocket

2 Offline: PC tool copies your videos or server titles onto the SD card

Set up a Jellyfin server at home

Jellyfin is free media-server software. If you already have a server, skip to the next part.

1. On a computer that stays on at home (a PC or a NAS), download Jellyfin Server from https://jellyfin.org/downloads and install it.
2. On that computer, open a web browser at `http://localhost:8096` and follow the setup: create your user name and password, and add your libraries (for example a **Movies** folder and a **Shows** folder).
3. Find the server's address on your home network. In the Jellyfin dashboard it is listed under **Networking**; it looks like `http://your-server:8096` (your computer's name or local address and port 8096).
4. Leave **transcoding** turned on (it is on by default): the server converts video into the small format the Chromatic and Pocket can show.
5. Keep the server on the same home network as the Chromatic.

Connect through a Chromatic

1. Install Jellymatic on the Chromatic and set it up there: Wi-Fi network, server address and sign-in (follow the Jellymatic instructions). Check that the Chromatic itself can browse your server.
2. Connect the Pocket and the Chromatic with a link cable.
3. On the Pocket, start the **Jellyfin** core. It looks for a Chromatic by itself every few seconds (or press **X** on the start screen).
4. On the Chromatic, open Jellyfin and choose **Pocket link** (in Jellymatic's menu).
5. The Pocket's header changes to **Chromatic connected** and your server's Home appears.

While connected, the Pocket quietly saves your Home, folders and posters. When you unplug the cable, that saved library stays browsable (titles that are not on the card are dimmed), and the app goes back to the Chromatic when you plug it in again.



What to expect

- The link cable is slow (a few tens of kilobytes per second). Pictures are about 128x72 and soft, and playback may pause now and then to refill its buffer.
- Browsing, posters and descriptions work well; the first visit to Home takes a little while, later visits come from the saved copy.
- For the best picture, copy titles to the card instead (Offline library).
- The connection uses only the link cable; the Pocket never sees your server address, password or tokens. Those stay on the Chromatic.

Optional: Remote Pilot

Remote Pilot is a free program for your Windows PC that shows the Chromatic's screen in a web browser, and lets you press its buttons, read its logs and save screenshots, over a USB cable or your home Wi-Fi. It works with a Chromatic running Jellymatic. You do not need it for PocketFin; it just makes the Chromatic side easier:

- **First-time setup from your desk**: go through the Chromatic's Wi-Fi, Jellyfin sign-in and Pocket link screens with the on-screen buttons, while watching the Chromatic's screen large on the PC.
- **Troubleshooting the link**: watch the Chromatic's screen and log on the PC while you hold the Pocket, instead of juggling two handhelds.
- **Cleaner bug reports**: its privacy eye (on by default) hides addresses and network names on the screen. Saved files are not masked, so check them before sharing.

- **Help from an AI assistant**: Claude, Codex or another assistant can drive the Chromatic through it to help you test, within the safety rules in its documentation.

It is not needed to play videos from the SD card.

Privacy

Your Wi-Fi details and server sign-in are set on the Chromatic (or, for the PC tool, in `private/jellyfin-connection.txt` on your PC, which Git ignores). Never post them in issues, logs or screenshots; see SECURITY.md.

Next: Offline library

Offline library on the card

Home > Offline library

- How it works
- Add your own videos
- Copy titles from your Jellyfin server
- Where the files go on the card
- Limits
- Troubleshooting

Only add videos you own or have the right to copy.

How it works

The Pocket cannot play ordinary video files. `tools/make_offline_library.py` converts them on your PC into the Pocket's format: 256x144 Motion-JPEG pictures at the video's own frame rate with IMA ADPCM sound. It writes a **library** file (Home rows, titles, descriptions, pictures) and one or more **pack** files (the videos). The Pocket's FPGA decodes the pictures.

You need Python, FFmpeg and PyAV on the PC: see What you need.

Add your own videos

1. Open a terminal (PowerShell) in the PocketFin folder.
2. Run, with your own file names:

   ```
   py -3 tools\make_offline_library.py --video "C:\Videos\My Video.mp4" --video
   "C:\Videos\Holiday 2024.mkv"
   ```

   Any format FFmpeg can read works (MP4, MKV, AVI, MOV, ...). The title shown on the Pocket is the file name without its extension. A picture from each video becomes its poster.

3. Wait: conversion takes roughly as long as the video, or less on a fast PC. It prints a summary at the end.
4. The result is in `build\offline-library\`. Copy **everything inside it** onto the root of the Pocket's SD card and let it replace the files it asks about.
5. Start the Jellyfin core, press **A**: your videos are in the **On this Pocket** tile in My Media.

A 60-second **Test Pattern** is added too, so the card always has something to play; add `--no-test-pattern` to leave it out.

Each run makes a **complete new library**: list every video you want on the card, not only new ones. The new library replaces the old one on the card.

Copy titles from your Jellyfin server

1. Copy `jellyfin-connection.example.txt` to `private\jellyfin-connection.txt` and fill in your server address (for example `http://your-server:8096`), user name and password, or an API key. This file stays on your PC; Git ignores the `private` folder.
2. Run:

   ```
   py -3 tools\make_offline_library.py --jellyfin --max-titles 4 --max-gb 8
   ```

   It takes titles from Continue Watching first, then the newest, up to the limits, and also stores your Home rows, folders, posters and descriptions. Titles that are listed but not copied show **Not on this Pocket**. You can combine `--jellyfin` with `--video`.
3. Copy `build\offline-library\` to the card as above.

The tool only reads from your server. It never changes watched status or playback positions there, and it prints only counts, never titles, addresses or tokens.

Where the files go on the card

On the card	What it is
`Assets/pocketlive/common/library.pjl`	The library (titles, rows, pictures)
`Assets/pocketlive/common/pack00.pjs` ... `pack11.pjs`	The videos

On the card	What it is
`Assets/pocketlive/ChaoticSPARTAN1.Jellyfin/Jellyfin.json`	The list of files the Pocket loads for this core

The Pocket loads only the packs listed in `Jellyfin.json`, so always copy it together with the packs. Old pack files that the new list does not name are ignored and can be deleted to save space.

Limits

- Up to **256 titles** and **12 packs** of up to **2 GiB** each (about 24 hours of video in total; about 0.9 GB per hour).
- One video must fit in one pack (up to about 2 hours 15 minutes).
- Frame rates: 23.976, 24, 25, 29.97 and 30 fps are kept as they are; 50 and 60 fps videos are halved.
- Subtitles are not shown. One audio track is used (the one FFmpeg picks by default).

Troubleshooting

Problem	What to do
`ffmpeg` or `ffprobe` not found	Install FFmpeg and open a new terminal; `ffmpeg -version` must work
`No module named 'av'`	`py -3 -m pip install av`
"Could not convert video N; skipped"	FFmpeg cannot read that file, or it has no video stream. Try playing it on the PC, or convert it to MP4 first
"Title exceeds a pack"	The video is too long for one pack; split it into parts
The Pocket shows only the old titles	`Jellyfin.json` was not copied, or the card was not ejected safely. Copy the whole `build\offline-library\` again
The core shows a black screen after copying	See Troubleshooting; put back the files from your card backup

Next: Troubleshooting and recovery

Troubleshooting and recovery

- Golden rules
- Black screen with a flashing backlight
- Go back to the previous files
- Other problems
- Remove the core cleanly
- Reporting a bug

PocketFin runs only while you have the core open. It cannot change the Pocket's firmware, and quitting the core or switching the Pocket off always returns it to normal.

Golden rules

- **Back up the card** before installing or updating.
- **Never edit** `interact.json` (or other files in `Cores/ChaoticSPARTAN1.Jellyfin/`). Changed versions of that file made the Pocket stop at a black screen during core start.
- Eject the card safely before taking it out of the computer.

Black screen with a flashing backlight

If the core stops at a black screen and the backlight flashes:

1. **Switch the Pocket off fully**: hold the power button until it turns off (about 10 seconds). A short press may not be enough.
2. Switch it on again. The Pocket starts normally; other cores are not affected.
3. Put back the PocketFin files you had before (next section), or reinstall `release/SD/` from a fresh download. Check that `Cores/ChaoticSPARTAN1.Jellyfin/interact.json` is the file from the release (`release/SHA256SUMS.txt` lists its checksum, which starts `4b53bc27`).

Go back to the previous files

- From your own card backup: copy back `Cores/ChaoticSPARTAN1.Jellyfin/`, `Assets/pocketlive/` and `Platforms/pocketlive.json`.
- If you used `Install-PocketFin.ps1`, the files it replaced are in `backups\<date-time>\` next to the script, in the same folder layout as the card. Copy them back onto the card.

Other problems

What you see	What to do
Jellyfin is not in the core list	`Cores/ChaoticSPARTAN1.Jellyfin/` or `Platforms/pocketlive.json` is missing; copy `release/SD/` again
The start screen says the JPEG decoder failed	The bitstream is damaged or from another version: copy `release/SD/` again
"Library on card: none"	Copy `release/SD-sample-library/` or your own library (see Offline library)
Your videos do not appear	Copy the whole library folder again, including `Assets/pocketlive/ChaoticSPARTAN1.Jellyfin/Jellyfin.json`
Resume points or the saved Chromatic library are lost	Quit the core from the Pocket's menu instead of switching off; check `Saves/pocketlive/common/` has the `pjcache*.sav` and `pjlive_log.sav` files (from `release/SD-first-install-saves/`)
The Chromatic is not found	Cable firmly in both ports; on the Chromatic choose **Pocket link**; press **X** on the Pocket's start screen
"Link fault" or titles do not start over the cable	Unplug and replug the cable, choose Pocket link again. Try another link cable
You cannot see what the Chromatic is doing while you hold the Pocket	Use Remote Pilot to watch the Chromatic's screen and log on your PC (more)

What you see	What to do
Playback over the cable pauses	Normal for the link's speed; copy the title to the card for smooth playback
Sound but no picture, or picture but no sound	Note what the start screen says and report it (below)

Remove the core cleanly

See Install > Remove PocketFin. Removing PocketFin leaves the rest of the card untouched.

Reporting a bug

The app keeps a short log in `Saves/pocketlive/common/pjlive_log.sav` (written when you quit the core). It is meant to hold timings and error messages only. Open an issue as described in CONTRIBUTING.md and attach the log if you can. Before posting anything, check it contains no Wi-Fi names, server addresses or tokens (SECURITY.md).

Next: FAQ

FAQ

- Basics
- Video and sound
- Jellyfin and the Chromatic
- Safety and legal

Basics

Why is the core called "Jellyfin" on the Pocket and "PocketFin" here? PocketFin is the project's name. The core keeps the name Jellyfin (folder `ChaoticSPARTAN1.Jellyfin`) so existing libraries and saves on cards keep working.

Do I need a Chromatic? No. Videos on the SD card play with nothing else. The Chromatic is only for live access to your server.

Does it work in the Analogue Dock? The core allows the Dock; it has not been tested much there.

Can I use the Pocket's Core Settings to change things? Not yet. Those screens show leftover template labels, which PocketFin ignores. See Using PocketFin.

Video and sound

Why 256x144? The FPGA decodes Motion-JPEG pictures in hardware. 256x144 at up to 30 fps keeps the files small (about 0.9 GB per hour) and the decoding smooth.

Will a 24 fps film play at 24 fps? Yes. Each title keeps its own frame rate (23.976, 24, 25, 29.97 or 30 fps). Press **R** while playing for film mode, which matches the screen refresh to the film.

Subtitles? Several audio languages? Not yet. One audio track is used.

Why is streaming through the Chromatic blurry? The link cable carries a few tens of kilobytes per second, so the server sends very small pictures. Copy titles to the card for the sharp version.

Jellyfin and the Chromatic

Does PocketFin need my Jellyfin password on the Pocket? No. The Pocket never sees your server address, password or tokens. The Chromatic signs in to your server (set up in Jellymatic), and the PC tool reads its own file on your PC.

Does it change my server? The PC tool only reads. Over the Chromatic, playback positions follow what Jellymatic does; see its documentation.

Is there a Wi-Fi accessory or a direct PC connection? No. The Pocket has no Wi-Fi, and cores cannot use its USB port. The two supported ways are the Chromatic link and copying to the card.

Which Chromatic software do I need? Jellymatic with its Pocket link option: https://github.com/ChaoticSPARTAN1/Jellymatic

Safety and legal

Can PocketFin damage my Pocket? It runs only while the core is open and does not touch the Pocket's firmware. The worst known case is a black screen at core start if core files are edited; switching the Pocket off fully and restoring the files fixes it (Troubleshooting).

Is this made by Analogue or Jellyfin? No. PocketFin is an independent project, not affiliated with Analogue or the Jellyfin project.

Can I share converted videos? Only if you have the right to. Only convert videos you own or may copy.

Is support guaranteed? No. Bug reports and pull requests are welcome, but answers are not guaranteed (CONTRIBUTING.md).

Next: Building from source

PocketFin / docs/Building-from-Source.md

Building from source

Home > Building from source

- What is in the repository
- Tools
- 1. Fetch the upstream sources
- 2. Build the app
- 3. Build the FPGA core
- 4. Make the release
- 5. Run the tests
- Offline documentation page

You do not need any of this to use PocketFin: `release/` is ready to copy. These steps rebuild every file in it from source.

What is in the repository

Folder	Contents
`pocket/`	The app (C): screens, library reader, playback, link-cable client, and PC test programs (`check_*.c`)
`shared/`	Headers shared with Jellymatic: wire framing, the request protocol, the A/V stream format
`rtl/`	PocketFin's FPGA blocks (link-cable byte engine, JPEG register interface, FIFOs) and their test benches
`core/`	`expected-sources.json` (the core's checked source list) and `generated/` (the CPU, boot ROM and kernel the release was built with)
`tools/`	Build, packaging, library and test scripts (Python and PowerShell)
`third_party/`	The Jellyfin logo files (CC BY-SA 4.0)

Folder	Contents
`release/`	The ready-to-copy SD card files

Big upstream projects are not copied in. `tools/fetch_upstreams.py` fetches them at the exact revisions in `upstream.lock.json`.

Tools

Windows 10/11 is assumed. Paths can be changed with environment variables (see `tools/toolpaths.py`).

Tool	Needed for	Where it is looked for
Git, Python 3.12+	everything	PATH (`py -3`)
RISC-V GCC 15.1 (`riscv64-unknown-elf-gcc` , MSYS2 UCRT64 packages listed in `upstream.lock.json`)	the app, the kernel	`POCKETFIN_RISCV_BIN` (default `local-tools\riscv\ucrt64\bin`)
MSYS2 (gcc, make, bash, libjpeg)	PC tests, kernel build	`POCKETFIN_MSYS` (default `C:\msys64`)
Intel Quartus Prime Lite 21.1.1 (free)	the FPGA core	`POCKETFIN_QUARTUS` (default `C:\intelFPGA_lite\21.1\quartus`)
Icarus Verilog 13	RTL tests	`POCKETFIN_IVERILOG` (folder with `bin\iverilog.exe`)
FFmpeg 7+ and PyAV	demo library, offline library, logo images	PATH; `py -3 -m pip install av`
fontTools	only to regenerate the font atlas	`py -3 -m pip install fonttools`

Tool	Needed for	Where it is looked for
Java 17 + sbt (downloaded by the script)	only to regenerate the CPU	`POCKETFIN_JAVA_HOME`

1. Fetch the upstream sources

```
py -3 tools\fetch_upstreams.py          # openfpgaSDK, openfpgaCore, core_jpeg into
upstream\
```

2. Build the app

```
.\tools\Build-LivePlayer.ps1            # -> build\pocket-live-player\pjlive.elf
```

With the pinned compiler the result is byte-identical to the released app (SHA-256 starts `e8ed291c`, see `release/SHA256SUMS.txt`).

The app's font is generated from Google's Roboto font. To regenerate it (optional; needs fontTools, `py -3 -m pip install fonttools`):

```
py -3 tools\fetch_upstreams.py --font   # Roboto 2.138, checked by SHA-256 ->
upstream\roboto\
py -3 tools\build_roboto_atlas.py       # -> pocket\fonts\roboto-medium-18.a8 and
.json
py -3 tools\build_ui_font.py            # -> pocket\ui_font.h and
pocket\ui_font_large.h
```

3. Build the FPGA core

```
py -3 tools\prepare_pocket_media_core.py    # pinned openfpgaCore + PocketFin's
changes -> build\pocket-media-core
.\tools\Build-PocketMediaCore.ps1 -Fit      # Quartus: 20-60 minutes -> ap_core.rbf
.\tools\Audit-PocketMediaCore.ps1           # checks timing of the clock-crossing
paths
```

`prepare_pocket_media_core.py` applies every change with exact-text checks, then compares the 43 files PocketFin adds or changes with `core/expected-sources.json` (the tree the

released bitstream was built from; PocketFin's own RTL files have since changed in comments only). What PocketFin changes in the runtime:

- the 33-bit link block is replaced by `pocket_media_accel` (byte link + JPEG decoder) on a 90 MHz CPU clock; the JPEG decoder runs on the existing 24.576 MHz clock;
- the register window grows by one address bit; the hardware/kernel contract number becomes `0x504a0001`, and the old link and network drivers stand down on this core;
- an upstream AXI write-address bug is fixed (the first delayed write beat was advanced by 4 bytes);
- the save fix in `core_bridge_cmd.v` keeps save-slot entries when the Pocket resets the core at quit (`tools/pocket_save_exit_patch.py`);
- a smaller single-issue CPU configuration (16 KiB / 64 KiB caches) and fitter seed 9.

The CPU (`VexiiRiscv_media.v`), boot ROM (`firmware.mif`) and kernel (`os.bin`) come from `core/generated/`. To rebuild them as well: `tools\fetch_upstreams.py --cpu`, `prepare_pocket_media_core.py --cpu-sources`, `tools\Build-MediaCPU.ps1`, `tools\Build-MediaFirmware.ps1`.

4. Make the release

```
py -3 tools\build_brand_assets.py       # Jellyfin logo -> core icon and platform
image (build\brand)
py -3 tools\build_sample_library.py     # 20-second generated test pattern
(build\sample-library)
py -3 tools\package_release.py          # -> release\SD, SD-sample-library, SD-first-
install-saves, SHA256SUMS.txt
```

`package_release.py` uses your own `ap_core.rbf` if you built one, otherwise the released bitstream. It refuses an `interact.json` other than the proven SDK template and checks the save-slot layout and the Chip32 loader.

5. Run the tests

All tests run on the PC; none touch a Pocket or a card.

```
py -3 tools\test_pocket_loader.py       # the loader writes saves back at quit
py -3 tools\test_link_gateway.py        # request/reply protocol, no secrets in
replies
py -3 tools\test_stream_worker.py       # streaming worker
py -3 tools\test_av_stream.py           # A/V stream format
py -3 tools\test_live_av.py             # encoder (FFmpeg + PyAV)
py -3 tools\test_sd_library.py          # on-card library end to end through the C
reader
py -3 tools\test_native_client.py       # the app's controller and screens; writes
build\native-client\ui-*.png
py -3 tools\test_pocket_save_exit.py    # the save fix on the real bridge RTL (Icarus;
Quartus sim library optional)
py -3 tools\test_rtl.py                 # link byte engine, FIFOs, JPEG decoder on the
app's self-test frame (Icarus)
```

`test_sd_library.py` needs `build\sample-library` (step 4). `test_pocket_save_exit.py` and `test_rtl.py` need `build\pocket-media-core` (step 3, prepare only).

Offline documentation page

`Open-Documentation.html` in the main folder holds every documentation page in one file that opens by double-click, with no server and no internet connection. **After editing any `.md` page, rebuild it:**

```
py -3 tools\build_docs_html.py          # rewrites Open-Documentation.html
py -3 tools\test_docs_html.py           # fails if it is out of date or would load
anything from the internet
```

It needs only Python (no packages). It converts README, `docs\*.md`, CONTRIBUTING, SECURITY, THIRD-PARTY-NOTICES and ADDITIONAL-TERMS (only files tracked by Git) and stores them inside the page; pictures load from `docs\images\`. A new page in `docs\` appears under **More** until you add it to a group in `GROUPS` at the top of the script. The README's banner and dividers are pictures in `docs\images\readme\` (dark and light versions), because GitHub does not show page styles. The same run writes `docs\index.html`, a copy for GitHub Pages (see below).

The PDF guide (`docs\PocketFin-Guide.pdf`, shown by GitHub in the browser) is printed from the same pages by Chrome or Edge, offline:

```
py -3 tools\build_docs_pdf.py           # rewrites docs\PocketFin-Guide.pdf and
docs\.guide-source.sha256
```

The test above says when the PDF is out of date (it compares `docs\.guide-source.sha256` with the pages and build scripts; it does not print again). Set `DOCS_BROWSER` if Chrome or Edge is not found.

For maintainers, when the repository is public: GitHub can host the documentation for free. In the repository's Settings > Pages, choose Deploy from a branch, branch `main`, folder `/docs`. The site is then `https://chaoticspartan1.github.io/PocketFin/` (it serves `docs\index.html`; `docs\.nojekyll` makes GitHub serve the files as they are). Add a link to it in the README only once it works.

Next: AI prompt examples

AI prompt examples

- Safety rules to include
- Building
- Troubleshooting
- Adding features
- Reviewing a change

You can use an AI coding assistant (for example Claude Code, OpenAI Codex or another assistant) to build, debug or extend PocketFin. Paste one of these prompts into the assistant while it is working in your copy of this repository. Check what it does; you are responsible for what ends up on your card.

Safety rules to include

Add this block to any prompt (or to the assistant's project instructions file):

```
Rules for this repository:
- Work on the PC only. Do not write to an SD card, a Pocket, a Chromatic or any device
unless I say so
  in this conversation, and never change console firmware.
- Never read, print or copy private/ (my Jellyfin sign-in) or anything with Wi-Fi
names, passwords,
  tokens or server addresses. Do not put secrets on command lines or in logs.
- Never edit release/SD/Cores/ChaoticSPARTAN1.Jellyfin/interact.json. Other versions
hung the Pocket.
- Keep the attribution "ChaoticSPARTAN1" in credits and the About screen (ADDITIONAL-
TERMS.md).
- Do not add third-party videos, images or code unless their licence allows it; say
which licence.
- Run the tests in docs/Building-from-Source.md after changes and tell me which passed
or failed.
- Do not push, publish or create repositories unless I ask.
```

Building

```
Read docs/Building-from-Source.md. Fetch the upstream sources, build the app with
tools/Build-LivePlayer.ps1 and tell me whether pjlive.elf matches the SHA-256 in
release/SHA256SUMS.txt.
If a tool is missing, tell me exactly what to install and where; do not install
anything without asking.
```

```
Prepare the FPGA core tree with tools/prepare_pocket_media_core.py and run a Quartus
map (no fit) with
tools/Build-PocketMediaCore.ps1. Summarise errors and warnings that concern files
under rtl/.
```

Troubleshooting

```
My Pocket shows a black screen with a flashing backlight when I start the Jellyfin
core. I already
switched it off fully. Here is the list of files in Cores/ChaoticSPARTAN1.Jellyfin on
my card with their
SHA-256 values: <paste>. Compare them with release/SHA256SUMS.txt and tell me which
files to restore.
Do not suggest editing interact.json.
```

```
tools/make_offline_library.py fails with this message: <paste, with any paths or names
removed>.
Find the cause in the code and suggest a fix. Test it with a video generated by FFmpeg
(testsrc2 + sine), not with my own files.
```

Adding features

```
I want subtitles for videos on the SD card. Read pocket/sd_library.h,
tools/pocket_library.py and
tools/live_av.py. Propose a design that keeps the existing library format readable
(version it), list
the files to change, then implement it with a PC test in pocket/check_sd_library.c.
Keep frame timing
and audio unchanged. Do not touch the FPGA core.
```

```
Add a Settings screen to the app for buffer length and volume, using the existing
values in
pocket/core_settings.h. Draw it in the same style as live_ui.h. Add a case to
pocket/check_live_controller.c and render a preview with tools/test_native_client.py.
Show me the
preview image.
```

Reviewing a change

```
Review my uncommitted changes for bugs. Focus on buffer sizes, integer overflow,
timing on the
Pocket's 90 MHz CPU and anything that could stop saves being written at quit. Run the
tests and report
results. Do not make changes; list findings with file and line.
```

Next: Credits and licences

Credits and licences

- Credits
- Licence
- Third-party parts
- Trademarks

Credits

PocketFin was created by ChaoticSPARTAN1.

It builds on the work of:

Project	By	Used for	Licence
openfpgaOS runtime and SDK	ThinkElastic	the FPGA system, OS kernel, app start-up code, templates	Apache-2.0
core_jpeg	Ultra-Embedded.com	the hardware JPEG decoder	Apache-2.0
VexiiRiscv / SpinalHDL	SpinalHDL	the RISC-V CPU	MIT (generated RTL)
Roboto font	Google	the app's text (glyph atlas generated by `tools/build_roboto_atlas.py`)	Apache-2.0
Jellyfin logo	the Jellyfin contributors	the logo in the app, core icon and platform image	CC BY-SA 4.0

Project	By	Used for	Licence
Analogue Pocket Framework	Analogue	the Pocket's core framework inside the bitstream	Analogue APF licence
Intel megafunctions	Intel	PLLs and memories inside the bitstream	Intel FPGA IP licence
musl libc	Rich Felker, et al. (the musl contributors)	C library in the app and kernel	MIT
cram0_phy	Adam Gastineau	memory interface inside the bitstream	MIT
dlmalloc, lib9660	Doug Lea; Erin Shepherd	memory allocator and ISO reader in the kernel	MIT-0; ISC

The app's About screen on the Pocket lists the main credits; the full list is in the notices on the card.

Licence

PocketFin's own code and files are licensed under the **GNU General Public License, version 3 or later** (LICENSE), with additional terms and one additional permission under section 7 (ADDITIONAL-TERMS.md):

- **Attribution (7(b))**: keep "ChaoticSPARTAN1" in the credits and in any About or credits screen.
- **Origin (7(c))**: do not remove that attribution or present the work as if it had no original author.
- **Bitstream permission**: you may convey the FPGA bitstream, which combines PocketFin with the Analogue Pocket Framework and Intel/Altera FPGA IP generated by Quartus; its Corresponding Source need not include the source of those parts, which keep their own licences.

Contributions are accepted under the same licence, additional terms and permission (CONTRIBUTING.md). PocketFin's patches to openfpgaCore's Apache-2.0 files are Apache-2.0, like the files they change.

Third-party parts

Every bundled or derived component, what it is in, and why it may be shipped: THIRD-PARTY-NOTICES.md. Licence texts, with each component's own copyright lines, and the openfpgaOS NOTICE files: LICENSES/. On the card the same notices are in `Notices/PocketLive/`.

The demo video is generated by this project (FFmpeg's test-pattern and tone generators); it contains no third-party footage or music. The openfpgaOS SDK's sample sound bank and music modules are not used.

Trademarks

Analogue and Analogue Pocket are trademarks of Analogue Enterprises Limited. Jellyfin is a trademark of its owners. ModRetro and Chromatic are trademarks of ModRetro, Inc. PocketFin is an independent project, not affiliated with, endorsed by or supported by any of them.

Back to Home

PocketFin / CONTRIBUTING.md

Contributing to PocketFin

Thank you for helping. Bug reports, bug fixes, documentation fixes and pull requests are welcome. PocketFin is a hobby project: there is no guarantee of a response or a fix, or of when.

Reporting a bug

Open an issue on GitHub with:

- what you did, what you expected and what happened;
- the PocketFin version (core.json, or the release you downloaded) and your Pocket firmware version;
- whether you used the card on its own or a Chromatic (and its Jellymatic version);
- what the start screen shows (the self-test lines);
- if you can, `Saves/pocketlive/common/pjlive_log.sav` from your card.

Never post Wi-Fi network names or passwords, server addresses, user names, API keys or tokens, or logs and screenshots that show them. Remove them first. Do not post copyrighted videos. Security problems: see SECURITY.md, not a public issue.

Pull requests

1. Keep changes small and focused, with a short description of why.
2. Run the tests in docs/Building-from-Source.md and say which you ran.
3. Do not change `release/` in the same pull request as source changes unless you rebuilt it with `tools/package_release.py`; say so if you did.
4. Never change `interact.json` in the release without a tested reason: other versions hung the Pocket.
5. New third-party code or assets must have a licence compatible with GPL-3.0-or-later; add them to THIRD-PARTY-NOTICES.md.
6. Keep the "ChaoticSPARTAN1" attribution (see ADDITIONAL-TERMS.md).

7. New source files start with `SPDX-License-Identifier: GPL-3.0-or-later` and the line `Additional terms and permissions: ADDITIONAL-TERMS.md`, like the existing ones.
8. After editing any documentation page, run `py -3 tools\build_docs_html.py` (Open-Documentation.html and docs/index.html) and `py -3 tools\build_docs_pdf.py` (the PDF guide); `tools\test_docs_html.py` checks all three. When the repository is public, GitHub Pages can serve the docs: Settings > Pages, branch `main`, folder `/docs`; the site is `https://chaoticspartan1.github.io/PocketFin/`.

By contributing you agree that your contribution is accepted under the same licence (GPL-3.0-or-later), with the same additional terms and the same additional permission as the rest of PocketFin (ADDITIONAL-TERMS.md). Changes to third-party files keep that file's own licence (for example, patches to openfpgaCore's Apache-2.0 files are Apache-2.0).

PocketFin / SECURITY.md

Security

Reporting a vulnerability

Please report security problems **privately**, not in a public issue: use GitHub's **Report a vulnerability** button on the repository's **Security** tab (private vulnerability reporting). Describe the problem and how to reproduce it. There is no guarantee of a response time.

Examples: a way for data on the link cable or in a library file to crash the app or run code, a tool that leaks a token or address, or a file that could harm a card.

Never post publicly

- Wi-Fi network names (SSIDs) or passwords
- Jellyfin server addresses, user names, user or item IDs
- passwords, API keys, access tokens or Quick Connect codes
- `private/jellyfin-connection.txt` or any file from `private/`
- logs, screenshots or photos that show any of the above

If you posted one by mistake, change the password or revoke the token in Jellyfin first, then edit the post.

What PocketFin keeps where

- The Pocket never receives your server address or sign-in. Library files on the card hold titles, descriptions, pictures and video only.
- The PC tools read your sign-in only from `private/jellyfin-connection.txt` on your PC (Git ignores the folder), never print it, and only read from your server.
- The Chromatic side (Jellymatic) keeps its own settings on the Chromatic; see its documentation.

PocketFin / THIRD-PARTY-NOTICES.md

Third-party notices

PocketFin's own code is GPL-3.0-or-later with additional terms and an additional permission for the FPGA bitstream (LICENSE, ADDITIONAL-TERMS.md). It is built with, and its release files contain, the parts below. Each keeps its own licence. The licence and notice files named below are in LICENSES/ in the source repository and, with the same file names, in `Notices/PocketLive/` on the SD card.

File paths such as `tools/...` or `pocket/...` refer to the source repository, https://github.com/ChaoticSPARTAN1/PocketFin . Paths starting with `Cores/`, `Assets/`, `Platforms/` or `Notices/` are on the SD card.

What is in each release file

Release file	What it contains	Licences
`Cores/ChaoticSPARTAN1.Jellyfin/os25.rbf_r` (FPGA bitstream)	openfpgaCore runtime RTL; PocketFin RTL (byte link, JPEG glue, FIFOs); core_jpeg decoder; VexiiRiscv CPU (generated); Analogue Pocket Framework files; Intel/Altera megafunction IP and Quartus library modules (PLLs, RAMs, DDIO); cram0_phy	Apache-2.0; GPL-3.0-or-later with the bitstream permission; Apache-2.0; MIT; Analogue APF licence; Intel FPGA IP licence; MIT
`Assets/pocketlive/common/os.bin` (OS kernel) and the boot ROM inside the bitstream	openfpgaCore firmware with PocketFin's small patches (Apache-2.0, see below); musl libc; dlmalloc; lib9660	Apache-2.0; MIT; MIT-0; ISC

Release file	What it contains	Licences
`Assets/pocketlive/common/pjlive.elf` (the app)	PocketFin app; Roboto Medium font atlas; Jellyfin logo; openfpgaOS SDK start-up code and headers; musl libc; libgcc	GPL-3.0-or-later; Apache-2.0; CC BY-SA 4.0; Apache-2.0; MIT; GPL-3.0 with GCC Runtime Library Exception
`Cores/.../loader.bin`	openfpgaOS SDK Chip32 loader with a short added tail (`tools/pocket_loader.py`)	Apache-2.0 (SDK part)
`Cores/.../*.json` , `Platforms/pocketlive.json`	openfpgaOS SDK templates, edited	Apache-2.0
`Cores/.../icon.bin` , `Platforms/_images/pocketlive.bin`	Jellyfin logo, scaled and converted to the Pocket's gray format	CC BY-SA 4.0
`Assets/pocketlive/common/library.pjl` , `pack00.pjs` (demo)	A test pattern and tone generated by FFmpeg's `testsrc2` and `sine` sources; no third-party footage or sound	GPL-3.0-or-later (generated by this project)

Components

openfpgaOS SDK - https://github.com/openfpgaOS/openfpgaSDK (commit a408ddc). Copyright (c) 2026 ThinkElastic. Apache-2.0 (Apache-2.0.txt); its NOTICE file is openfpgaSDK-NOTICE.txt. musl libc inside it: see below. The SDK also contains a SoundFont and tracker modules of unclear provenance (its `LicenseRef-Proprietary-SampleData` and `LicenseRef-ThirdParty-Media`); PocketFin does **not** use or ship any of them.

openfpgaCore - https://github.com/openfpgaOS/openfpgaCore (commit 618a3eb). Copyright (c) 2026 ThinkElastic. Apache-2.0 for its own files (Apache-2.0.txt); its NOTICE file is

openfpgaCore-NOTICE.txt. PocketFin does not include this source; it is fetched by `tools/fetch_upstreams.py` and patched by `tools/prepare_pocket_media_core.py`.

PocketFin's changes to openfpgaCore's Apache-2.0 files (the patch hunks applied to `src/firmware/os/hal/link.c`, `src/firmware/os/hal/net.c`, `src/firmware/os/hal/regs.h` and `src/fpga/common/axi_periph_slave.v` by `tools/prepare_pocket_media_core.py`) are contributed under the Apache License, Version 2.0, so those patched files, and the kernel and boot ROM built from them, stay under Apache-2.0. The rest of that script is GPL-3.0-or-later like the rest of PocketFin.

The bitstream built from openfpgaCore also contains:

- the **Analogue Pocket Framework** (`apf/`, `core_top.v`, `core_bridge_cmd.v`, `io_sdram.v`), Copyright (c) Analogue Enterprises Limited, under the Analogue Pocket Framework Software License Agreement (LicenseRef-Analogue-Pocket-Framework.txt). That agreement says that where the GPL applies and its terms are inconsistent with the agreement, the GPL prevails. PocketFin changes `core_bridge_cmd.v` (save-on-exit fix) and `core_top.v` (link block) by script; the changes are in `tools/pocket_save_exit_patch.py` and `tools/prepare_pocket_media_core.py`.
- **Intel/Altera megafunction IP** generated by Quartus (PLLs, block RAM, DDIO) and Quartus library modules, Copyright (c) Intel Corporation, under the licence in LicenseRef-Intel-FPGA-IP.txt: it may be used to program Intel/Altera devices, which is what the Pocket's FPGA is.
- `cram0_phy.sv`, Copyright (c) 2022 Adam Gastineau, MIT (MIT-cram0_phy.txt).

PocketFin's licence grants an additional permission to convey the bitstream with the Analogue Pocket Framework and the Intel/Altera IP (ADDITIONAL-TERMS.md). The Analogizer and other third-party video files in openfpgaCore are **not** part of this build (Quartus' list of source files read for the released bitstream does not include them).

The kernel (`os.bin`) and the boot ROM also include:

- **musl libc** 1.2.5, Copyright (c) 2005-2020 Rich Felker, et al., MIT; musl's full COPYRIGHT file is MIT-musl-COPYRIGHT.txt. The app (`pjlive.elf`) links the musl libc that ships in the openfpgaOS SDK, under the same licence.
- **dlmalloc**, Copyright 2023 Doug Lea, MIT-0 (MIT-0-dlmalloc.txt).
- **lib9660**, Copyright (c) 2014 Erin Shepherd, ISC (ISC-lib9660.txt).

VexiiRiscv - https://github.com/openfpgaOS/VexiiRiscv (commit 580b76c), Copyright (c) 2023 SpinalHDL, MIT (MIT-VexiiRiscv.txt; also `core/generated/VexiiRiscv-LICENSE.txt`). `core/generated/VexiiRiscv_media.v` was generated from it with **SpinalHDL** (commit 6f8510c; core LGPL-3.0, library MIT; SpinalHDL states that generated RTL belongs to the user) and rvls (commit 94f850e).

core_jpeg - https://github.com/ultraembedded/core_jpeg (commit bb03cce), Copyright 2020 Ultra-Embedded.com, Apache-2.0 (Apache-2.0.txt). PocketFin's port removes an unused RAM write port in four files and qualifies the input stage's marker detection; both changes are marked `LOCAL` in the generated files and listed in `tools/prepare_pocket_media_core.py`, and they are contributed under Apache-2.0 like the files they change. The source is fetched, not included.

Roboto (font) - Roboto Medium, version 2.138, Copyright 2011 Google Inc., Apache-2.0 (Apache-2.0.txt), from https://github.com/googlefonts/roboto-2 (release v2.138; the download and file are pinned by SHA-256 in `upstream.lock.json`). The app's font atlas (`pocket/fonts/roboto-medium-18.a8`) is rasterized directly from the font file by `tools/build_roboto_atlas.py`, and `tools/build_ui_font.py` scales it into `pocket/ui_font.h` and `pocket/ui_font_large.h`. These generated files stay under Apache-2.0; what was changed is described in the font's notice.

Jellyfin logo - from https://github.com/jellyfin/jellyfin-ux (commit 69a2b7b), Copyright the Jellyfin contributors, CC BY-SA 4.0 (the logo's licence). Scaled and converted to the Pocket's formats by `tools/build_brand_assets.py` (`pocket/ui_brand.h`, the core icon and the platform image); no other change. Those adapted images are under CC BY-SA 4.0 as well. "Jellyfin" is a trademark of its owners; PocketFin is not affiliated with or endorsed by the Jellyfin project.

GCC runtime (libgcc) - linked into the app by the RISC-V compiler; GPL-3.0 with the GCC Runtime Library Exception, which allows this use.

Build tools (not shipped)

Intel Quartus Prime Lite, the RISC-V GCC toolchain, MSYS2, Icarus Verilog, Python, fontTools, PyAV and FFmpeg are used to build and test but are not part of the release files. FFmpeg's `testsrc2` and `sine` sources generate the demo video; the result contains no FFmpeg code.

Trademarks

Analogue and Analogue Pocket are trademarks of Analogue Enterprises Limited. Jellyfin is a trademark of its owners. ModRetro and Chromatic are trademarks of ModRetro, Inc. They are used here only to say what PocketFin works with. PocketFin is an independent project, not affiliated with Analogue, ModRetro or the Jellyfin project.

PocketFin / ADDITIONAL-TERMS.md

Additional terms and permissions (GNU GPL version 3, section 7)

PocketFin is free software under the GNU General Public License, version 3 or (at your option) any later version. The full licence is in LICENSE.

The following additional permission and additional terms apply to the parts of PocketFin written by its author, ChaoticSPARTAN1 (and to contributions accepted under the same terms, see CONTRIBUTING.md).

Additional permission for the FPGA bitstream (section 7)

> If you modify this Program, or any covered work, by combining or linking it with the Analogue Pocket Framework (or a modified version of it) or with Intel/Altera FPGA IP generated by Intel Quartus (including megafunctions such as PLLs, RAM, FIFOs and DDIO, and Quartus library modules), the licensors of this Program grant you additional permission to convey the resulting work, including FPGA bitstreams. Corresponding Source for a non-source form of such a combination need not include the source code of those parts, which remain under their own licences.

As section 7 of the GPL allows, anyone who conveys a copy may remove this additional permission from that copy (and from works based on it).

Additional terms (sections 7(b) and 7(c))

> **Author attribution (section 7(b)).** If you convey PocketFin or a work based on it, in source or in binary form, you must keep the author attribution "ChaoticSPARTAN1" (for example "Created by ChaoticSPARTAN1") in the credits section of the README or equivalent documentation, and in any About, Credits or similar screen that the program displays. You may add your own name next to it.
>
> **Origin (section 7(c)).** You may not remove that attribution or otherwise misrepresent the origin of the material, for example by presenting the work as if it had no original author.

The first term is a requirement to preserve a reasonable author attribution in the sense of section 7(b); the second prohibits misrepresentation of the origin of the material in the sense of section 7(c). Neither restricts anything else the GPL allows. If you receive a copy where this notice was removed, these terms still apply.

Third-party parts keep their own licences and notices; see THIRD-PARTY-NOTICES.md.